

```

/**
 * Keystone backend: owner-only admin, Stripe subscription gating, hot leads.
 * Assumes Hono, better-auth (auth.api.getSession) and Drizzle + SQLite.
 * Adjust the three imports below to match your project.
 * Mount with: app.route("/", keystone)
 */
import { Hono, type MiddlewareHandler } from "hono";
import { and, desc, eq, sql } from "drizzle-orm";
import Stripe from "stripe";
import { db } from "../db";
import { auth } from "../auth";
import { aiUsage, hotLeads, leads, prospects, stripeEvents, subscriptions } from

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);
const OWNER_EMAIL = (process.env.OWNER_EMAIL ?? "").trim().toLowerCase();
const APP_URL = process.env.APP_URL ?? "http://localhost:3000";

type Env = { Variables: { user: { id: string; email: string }; isAdmin: boolean } };
export const keystone = new Hono<Env>();

/* ----- middleware ----- */

/** Signed-in user. Admin = the one email in OWNER_EMAIL (server-side, not a client)
const requireUser: MiddlewareHandler<Env> = async (c, next) => {
  const session = await auth.api.getSession({ headers: c.req.raw.headers });
  if (!session?.user) return c.json({ error: "Sign in required" }, 401);
  const email = session.user.email.toLowerCase();
  c.set("user", { id: session.user.id, email });
  c.set("isAdmin", OWNER_EMAIL !== "" && email === OWNER_EMAIL);
  await next();
};

const requireAdmin: MiddlewareHandler<Env> = async (c, next) => {
  if (!c.get("isAdmin")) return c.json({ error: "Owner only" }, 403);
  await next();
};

/** Paywall: owner always passes; everyone else needs an active or trialing subscription
const requireSub: MiddlewareHandler<Env> = async (c, next) => {
  if (c.get("isAdmin")) return next();
  const [s] = await db.select().from(subscriptions).where(eq(subscriptions.userId
  if (!s || ![ "active", "trialing" ].includes(s.status)) {
    return c.json({ error: "Active subscription required", upgrade: "/app/billing

```

```

    }
    await next();
  };

  /* ----- billing ----- */

  async function customerFor(user: { id: string; email: string }) {
    const [row] = await db.select().from(subscriptions).where(eq(subscriptions.user
    if (row) return row.stripeCustomerId;
    const customer = await stripe.customers.create({ email: user.email, metadata: {
    await db.insert(subscriptions).values({ userId: user.id, stripeCustomerId: cust
    return customer.id;
  }

  const PRICES: Record<string, string | undefined> = {
    basic: process.env.STRIPE_PRICE_BASIC,
    pro: process.env.STRIPE_PRICE_PRO,
    proplus: process.env.STRIPE_PRICE_PROPLUS,
  };

  keystone.post("/api/billing/checkout", requireUser, async (c) => {
    const user = c.get("user");
    const { plan } = await c.req.json().catch(() => ({ plan: "pro" }));
    const price = PRICES[plan as string];
    if (!price) return c.json({ error: "Unknown plan. Use basic, pro or proplus." });
    const [prev] = await db.select().from(subscriptions).where(eq(subscriptions.use
    const trialDays = prev?.stripeSubscriptionId ? 0 : Number(process.env.TRIAL_DAY
    const session = await stripe.checkout.sessions.create({
      mode: "subscription",
      customer: await customerFor(user),
      line_items: [{ price, quantity: 1 }],
      client_reference_id: user.id,
      subscription_data: { metadata: { userId: user.id }, ...(trialDays > 0 ? { tri
      allow_promotion_codes: true,
      success_url: `${APP_URL}/app/billing?status=success`,
      cancel_url: `${APP_URL}/app/billing?status=cancelled`,
    });
    return c.json({ url: session.url });
  });

  keystone.post("/api/billing/portal", requireUser, async (c) => {
    const [s] = await db.select().from(subscriptions).where(eq(subscriptions.userId
    if (!s) return c.json({ error: "No billing account yet" }, 404);
    const portal = await stripe.billingPortal.sessions.create({
      customer: s.stripeCustomerId,
      return_url: `${APP_URL}/app/billing`,

```

```

    });
    return c.json({ url: portal.url });
});

keystone.get("/api/billing/status", requireUser, async (c) => {
    const [s] = await db.select().from(subscriptions).where(eq(subscriptions.userId
    return c.json({
        isAdmin: c.get("isAdmin"),
        status: s?.status ?? "none",
        currentPeriodEnd: s?.currentPeriodEnd ?? null,
        active: c.get("isAdmin") || ["active", "trialing"].includes(s?.status ?? ""),
    });
});

async function syncSubscription(sub: Stripe.Subscription) {
    const userId = sub.metadata?.userId;
    if (!userId) return;
    const item: any = sub.items.data[0];
    const end = item?.current_period_end ?? (sub as any).current_period_end;
    const values = {
        stripeCustomerId: typeof sub.customer === "string" ? sub.customer : sub.custo
        stripeSubscriptionId: sub.id,
        status: sub.status,
        plan: item?.price?.metadata?.keystone_plan ?? "pro",
        currentPeriodEnd: end ? new Date(end * 1000) : null,
        updatedAt: new Date(),
    };
    await db.insert(subscriptions).values({ userId, ...values }).onConflictDoUpdate
}

/** Stripe calls this. No auth middleware: the signature is the auth. Needs the r
keystone.post("/api/stripe/webhook", async (c) => {
    const signature = c.req.header("stripe-signature");
    if (!signature) return c.json({ error: "Missing signature" }, 400);
    let event: Stripe.Event;
    try {
        event = stripe.webhooks.constructEvent(await c.req.text(), signature, process
    } catch {
        return c.json({ error: "Bad signature" }, 400);
    }
    const fresh = await db.insert(stripeEvents).values({ id: event.id }).onConflict
    if (fresh.length === 0) return c.json({ received: true, duplicate: true });

    if (event.type === "checkout.session.completed") {
        const s = event.data.object as Stripe.Checkout.Session;
        if (s.mode === "subscription" && s.subscription) {

```

```

        const id = typeof s.subscription === "string" ? s.subscription : s.subscrip
        await syncSubscription(await stripe.subscriptions.retrieve(id));
    }
} else if (
    event.type === "customer.subscription.created" ||
    event.type === "customer.subscription.updated" ||
    event.type === "customer.subscription.deleted"
) {
    await syncSubscription(event.data.object as Stripe.Subscription);
}
return c.json({ received: true });
});

/* ----- hot leads (subscribers) ----- */

export const SIGNALS = ["no_website", "not_mobile", "no_https", "old_copyright",
const clean = (v: unknown, max = 300) => (typeof v === "string" && v.trim() ? v.t
const scoreOf = (signals: unknown) =>
    Array.isArray(signals) ? [...new Set(signals.filter((s) => (SIGNALS as readonly

keystone.get("/api/hot-leads", requireUser, requireSub, async (c) => {
    const rows = await db.select().from(hotLeads).where(eq(hotLeads.userId, c.get("
    return c.json(rows.map((r) => ({ ...r, hot: r.score >= 4 })));
});

keystone.post("/api/hot-leads", requireUser, requireSub, async (c) => {
    const b = await c.req.json().catch(() => ({}));
    const businessName = clean(b.businessName, 120);
    if (!businessName) return c.json({ error: "businessName is required" }, 400);
    const signals = scoreOf(b.signals);
    const [row] = await db.insert(hotLeads).values({
        userId: c.get("user").id,
        businessName,
        contactName: clean(b.contactName),
        email: clean(b.email),
        phone: clean(b.phone, 40),
        website: clean(b.website, 500),
        industry: clean(b.industry, 80),
        city: clean(b.city, 80),
        signals,
        score: signals.length,
    }).returning();
    return c.json(row, 201);
});

keystone.delete("/api/hot-leads/:id", requireUser, requireSub, async (c) => {

```

```

    await db.delete(hotLeads).where(and(eq(hotLeads.id, Number(c.req.param("id"))),
    return c.json({ ok: true }));
});

/** Move a hot lead into the sales pipeline (prospects table). */
keystone.post("/api/hot-leads/:id/promote", requireUser, requireSub, async (c) =>
    const userId = c.get("user").id;
    const [l] = await db.select().from(hotLeads).where(and(eq(hotLeads.id, Number(c
    if (!l) return c.json({ error: "Not found" }, 404);
    const [p] = await db.insert(prospects).values({
        userId, businessName: l.businessName, contactName: l.contactName, email: l.em
        website: l.website, industry: l.industry, city: l.city, stage: "new",
        notes: `Hot lead score ${l.score}/6: ${l.signals.join(", ") || "none"}` ,
    }).returning();
    await db.update(hotLeads).set({ status: "promoted", updatedAt: new Date() }).wh
    return c.json(p, 201);
});

/* ----- owner-only admin ----- */

const admin = new Hono<Env>();
admin.use("*", requireUser, requireAdmin);

admin.get("/overview", async (c) => {
    const subs = await db.select({ status: subscriptions.status, n: sql<number>`cou
    const [hot] = await db.select({ n: sql<number>`count(*)` }).from(hotLeads);
    const [inbound] = await db.select({ n: sql<number>`count(*)` }).from(leads).whe
    const [won] = await db.select({ total: sql<number>`coalesce(sum(${prospects.dea
    return c.json({ subscribers: subs, hotLeads: hot.n, newInboundLeads: inbound.n,
});

admin.get("/subscribers", async (c) => c.json(await db.select().from(subscription
admin.get("/hot-leads", async (c) => c.json(await db.select().from(hotLeads).orde
admin.get("/leads", async (c) => c.json(await db.select().from(leads).orderBy(des
admin.patch("/leads/:id", async (c) => {
    const { status } = await c.req.json().catch(() => ({}));
    if (!["new", "contacted", "won", "lost"].includes(status)) return c.json({ erro
    await db.update(leads).set({ status }).where(eq(leads.id, Number(c.req.param("i
    return c.json({ ok: true });
});

keystone.route("/api/admin", admin);

/* ----- AI module (subscribers) ----- */

```

```

const DAILY_LIMIT: Record<string, number> = { basic: 20, pro: 100, proplus: 300 }
const AI_MODEL = process.env.AI_MODEL ?? "claude-sonnet-5-5";
const JSON_ONLY = "Return ONLY valid JSON, no markdown fences, no commentary. ";
const NO_HYPE = "Do not promise rankings, guaranteed results, lower ad costs, or
const SYSTEM: Record<string, string> = {
  leads: JSON_ONLY + "You find sales leads for a small web design agency. Use web
  ads: JSON_ONLY + NO_HYPE + "Write ads for the request. Return {\`google\`: {\`he
  video: JSON_ONLY + NO_HYPE + "Plan a 15 to 45 second vertical video ad. You wri
  design: JSON_ONLY + "You restyle a small business website. The context holds th
  coach: NO_HYPE + "You are the personal business assistant for the owner of AIU,
  write: NO_HYPE + "You write clear, friendly website and marketing copy for smal
};

```

```

keystone.post("/api/ai", requireUser, requireSub, async (c) => {
  const body = await c.req.json().catch(() => ({}));
  const mode = String(body.mode ?? "");
  const prompt = String(body.prompt ?? "").trim().slice(0, 1500);
  const context = String(body.context ?? "").slice(0, 3000);
  const history: { role: "user" | "assistant"; content: string }[] =
    mode === "coach" && Array.isArray(body.history)
      ? body.history.filter((m: any) => (m?.role === "user" || m?.role === "assis
        .slice(-8).map((m: any) => ({ role: m.role, content: m.content.slice(0,
          : []);
  while (history.length && history[0].role === "assistant") history.shift();
  if (!SYSTEM[mode] || !prompt) return c.json({ error: "Pick a mode and enter a p
  if (!process.env.ANTHROPIC_API_KEY) return c.json({ error: "AI is not configure

  const user = c.get("user");
  if (!c.get("isAdmin")) {
    const [s] = await db.select().from(subscriptions).where(eq(subscriptions.user
    const limit = DAILY_LIMIT[s?.plan ?? "basic"] ?? 20;
    const day = new Date().toISOString().slice(0, 10);
    const [u] = await db.select().from(aiUsage).where(and(eq(aiUsage.userId, user
    if ((u?.n ?? 0) >= limit) return c.json({ error: `Daily AI limit reached (${1
    await db.insert(aiUsage).values({ userId: user.id, day, n: 1 })
      .onConflictDoUpdate({ target: [aiUsage.userId, aiUsage.day], set: { n: sql`
  }

  try {
    const r = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "x-api-key": process.env.ANTHROPIC_API_KEY, "anthropic-version":
      body: JSON.stringify({
        model: AI_MODEL,
        max_tokens: 2500,
        system: SYSTEM[mode],

```

```

        messages: [...history, { role: "user", content: context ? `Context:\n${co
        ...(mode === "leads" ? { tools: [{ type: "web_search_20250305", name: "we
    }},
    });
    if (!r.ok) {
        console.error("AI error", r.status, (await r.text()).slice(0, 300));
        return c.json({ error: "The AI service had a problem. Try again in a moment
    }
    const data: any = await r.json();
    const text = (data.content ?? []).filter((b: any) => b.type === "text").map((
    return c.json({ mode, text });
    } catch (e) {
        console.error("AI failure", e);
        return c.json({ error: "The AI service could not be reached." }, 502);
    }
    });

/* ----- public "get a demo" form ----- */

const leadHits = new Map<string, number[]>();
keystone.post("/api/leads", async (c) => {
    const ip = c.req.header("x-forwarded-for")?.split(",")[0].trim() ?? "unknown";
    const now = Date.now();
    const recent = (leadHits.get(ip) ?? []).filter((t) => now - t < 3_600_000);
    if (recent.length >= 5) return c.json({ error: "Too many requests. Please try a
    leadHits.set(ip, [...recent, now]);
    const b = await c.req.json().catch(() => ({}));
    const name = clean(b.name, 100), email = clean(b.email, 200), interest = clean(
    if (!name || !email || !/^S+@\S+\.\S+$/ .test(email) || !interest) return c.jso
    await db.insert(leads).values({ name, email, company: clean(b.company, 120), in
    return c.json({ ok: true }, 201);
    });

```